

Rust

Perfectionnement

Haute Performance

Rust Haute Performance

Programmation Concurrente Avancée

Agenda du Jour 2

Horaire	Module	Durée
09:00 – 11:00	Module 2.1 – Primitives de synchronisation	2h
11:00 – 13:00	Module 2.2 – Programmation atomique	2h
14:00 – 16:00	Module 2.3 – Structures lock-free	2h
16:00 – 17:00	Module 2.4 – Pattern Actor	1h

“ **Objectif** : Concevoir des composants concurrents sûrs et performants, des Mutex aux structures lock-free ”

Module 2.1 – Primitives de Synchronisation

Mutex, RwLock, Condvar

Mutex : Contention et stratégies

```
use std::sync::Mutex;
use parking_lot::Mutex as FastMutex; // Alternative plus performante

// std::sync::Mutex : poison on panic, OS-backed
let data = Mutex::new(HashMap::new());
{
    let mut guard = data.lock().unwrap(); // Peut poisonner
    guard.insert("key", "value");
} // guard droppé → unlock automatique

// parking_lot::Mutex : pas de poison, spin-then-park
let data = FastMutex::new(HashMap::new());
{
    let mut guard = data.lock(); // Pas de Result
    guard.insert("key", "value");
}
```

Mutex : Contention et stratégies

Caractéristique	<code>std::Mutex</code>	<code>parking_lot::Mutex</code>
Taille	40 bytes	1 byte
Poison	Oui	Non
Stratégie	OS futex	Spin-then-park
Performance (faible contention)	Bonne	Meilleure

RwLock : Lecteurs multiples

```
use parking_lot::RwLock;

struct ConfigStore {
    config: RwLock<AppConfig>,
}

impl ConfigStore {
    // Multiples lecteurs simultanés
    fn get_port(&self) -> u16 {
        self.config.read().server.port
    }
    // Un seul writer, bloque les readers
    fn update(&self, new_config: AppConfig) {
        let mut guard = self.config.write();
        *guard = new_config;
    }
}
```

Condvar : Producer/Consumer

```
use std::sync::{Mutex, Condvar, Arc};

struct BoundedQueue<T> {
    data: Mutex<VecDeque<T>>,
    not_empty: Condvar,
    not_full: Condvar,
    capacity: usize,
}

impl<T> BoundedQueue<T> {
    fn push(&self, item: T) {
        let mut data = self.data.lock().unwrap();
        while data.len() >= self.capacity {
            data = self.not_full.wait(data).unwrap(); // Attend qu'il y ait de la place
        }
        data.push_back(item);
        self.not_empty.notify_one(); // Réveille un consommateur
    }

    fn pop(&self) -> T {
        let mut data = self.data.lock().unwrap();
        while data.is_empty() {
            data = self.not_empty.wait(data).unwrap(); // Attend un élément
        }
        let item = data.pop_front().unwrap();
        self.not_full.notify_one(); // Réveille un producteur
        item
    }
}
```


Atelier : Buffer circulaire thread-safe

Objectif

Implémenter un ring buffer thread-safe FIFO multi-producer, multi-consumer

Tâches

1. Capacité fixe, FIFO
2. `push()` bloque si plein
3. `pop()` bloque si vide
4. `try_push()` / `try_pop()` non-bloquants
5. Multi-producer, multi-consumer
6. Benchmark vs crossbeam channel

Temps : 30 minutes

Module 2.2 – Programmation Atomique

Memory orderings et CAS

Memory Orderings

```
use std::sync::atomic::{AtomicU64, AtomicBool, Ordering};
// Relaxed : pas de garantie d'ordre (compteurs, statistiques)
static COUNTER: AtomicU64 = AtomicU64::new(0);
COUNTER.fetch_add(1, Ordering::Relaxed);
// Acquire/Release : synchronisation producer/consumer
static DATA_READY: AtomicBool = AtomicBool::new(false);
static DATA: AtomicU64 = AtomicU64::new(0);

// Producer
DATA.store(42, Ordering::Relaxed);
DATA_READY.store(true, Ordering::Release); // Tout avant est visible après Acquire

// Consumer
while !DATA_READY.load(Ordering::Acquire) {} // Attend Release
let value = DATA.load(Ordering::Relaxed);    // Garanti de voir 42

// SeqCst : ordre total global (le plus cher, le plus sûr)
COUNTER.fetch_add(1, Ordering::SeqCst);
```

Orderings : Guide de choix

Ordering	Performance	Garantie	Cas d'usage
Relaxed	★★★★★	Aucun ordre	Compteurs, métriques
Acquire	★★★★☆	Voit les stores Release	Consumer side
Release	★★★★☆	Visible par Acquire	Producer side
AcqRel	★★★☆☆	Acquire + Release	Read-modify-write
SeqCst	★★☆☆☆	Ordre total global	Quand en doute

Règle OVH : Relaxed pour les métriques, SeqCst par défaut, optimiser ensuite si le profiling montre un bottleneck

Compare-and-Swap (CAS)

```
use std::sync::atomic::{AtomicU64, Ordering};
/// Incrémente atomiquement avec CAS (lock-free)
fn atomic_max(current: &AtomicU64, new_val: u64) {
    let mut old = current.load(Ordering::Relaxed);
    loop {
        if old >= new_val { return; }
        match current.compare_exchange_weak(
            old,                // Valeur attendue
            new_val,            // Nouvelle valeur
            Ordering::AcqRel,    // Succès
            Ordering::Relaxed,   // Échec
        ) {
            Ok(_) => return,    // CAS réussi
            Err(actual) => old = actual, // Retry avec la valeur actuelle
        }
    }
}

// Usage : mise à jour du max latence sans lock
static MAX_LATENCY: AtomicU64 = AtomicU64::new(0);
atomic_max(&MAX_LATENCY, measured_latency_ns);
```

Atelier : Compteur de références atomique

Objectif

Implémenter un `MyArc<T>` simplifié

Tâches

1. Compteur de références atomique
2. `clone()` incrémente (Relaxed)
3. `drop()` décrémente (AcqRel) et libère si 0
4. `deref()` pour accès aux données
5. Tests multi-threadés

Temps : 30 minutes

Module 2.3 – Structures Lock-Free

Crossbeam : Queues lock-free

```
use crossbeam::queue::{ArrayQueue, SegQueue};
use crossbeam::deque::{Worker, Stealer, Steal};
// ArrayQueue : bounded, MPMC, lock-free
let q = ArrayQueue::new(1024);
q.push(42).unwrap();
let val = q.pop(); // Some(42)
// SegQueue : unbounded, MPMC, lock-free
let q = SegQueue::new();
q.push("hello");
let val = q.pop(); // Some("hello")
// Work-stealing deque (comme Tokio scheduler)
let worker = Worker::new_fifo();
let stealer = worker.stealer();
worker.push(Task::new(1));
worker.push(Task::new(2));

// Worker prend du bas
let task = worker.pop(); // Task(2)
// Stealer vole du haut
match stealer.steal() {
    Steal::Success(task) => process(task),
    Steal::Empty => { /* rien à voler */ }
    Steal::Retry => { /* contention, réessayer */ }
}
```


Epoch-based Reclamation

```
use crossbeam::epoch::{self, Atomic, Owned, Shared};
use std::sync::atomic::Ordering;
struct LockFreeStack<T> {
    head: Atomic<Node<T>>,
}
struct Node<T> {
    data: T,
    next: Atomic<Node<T>>,
}
impl<T> LockFreeStack<T> {
    fn push(&self, data: T) {
        let new_node = Owned::new(Node {
            data,
            next: Atomic::null(),
        });

        let guard = epoch::pin(); // Protège contre la réclamation
        loop {
            let head = self.head.load(Ordering::Relaxed, &guard);
            new_node.next.store(head, Ordering::Relaxed);

            match self.head.compare_exchange(
                head, new_node,
                Ordering::Release, Ordering::Relaxed, &guard
            ) {
                Ok(_) => return,
                Err(e) => { /* new_node est retourné, retry */ }
            }
        }
    }
}
```

Concurrent HashMap (DashMap)

```
use dashmap::DashMap;
// Thread-safe HashMap sans lock global
let map: DashMap<String, Vec<Metric>> = DashMap::new();
// Insertion concurrente (lock par shard)
map.entry("server-01".to_string())
    .or_insert_with(Vec::new)
    .push(Metric::new());
// Lecture concurrente
if let Some(metrics) = map.get("server-01") {
    println!("Metrics: {:?}", metrics.value());
}
// Itération concurrente (snapshot)
for entry in map.iter() {
    println!("{key}: {value} metrics", entry.key(), entry.value().len());
}
```

Cas OVH : Stockage in-memory pour caches partagés, stats par hostname/service

Module 2.4 – Pattern Actor

Acteur avec tokio::mpsc

```
use tokio::sync::{mpsc, oneshot};

// Messages de l'acteur
enum CacheMsg {
    Get { key: String, reply: oneshot::Sender<Option<String>> },
    Set { key: String, value: String },
    Delete { key: String },
}

// Acteur : boucle de traitement de messages
async fn cache_actor(mut rx: mpsc::Receiver<CacheMsg>) {
    let mut store = HashMap::new();

    while let Some(msg) = rx.recv().await {
        match msg {
            CacheMsg::Get { key, reply } => {
                let _ = reply.send(store.get(&key).cloned());
            }
            CacheMsg::Set { key, value } => {
                store.insert(key, value);
            }
            CacheMsg::Delete { key } => {
                store.remove(&key);
            }
        }
    }
}

// Handle pour communiquer avec l'acteur
#[derive(Clone)]
struct CacheHandle {
    tx: mpsc::Sender<CacheMsg>,
}

impl CacheHandle {
    fn new(buffer: usize) -> Self {
        let (tx, rx) = mpsc::channel(buffer);
        tokio::spawn(cache_actor(rx));
        Self { tx }
    }

    async fn get(&self, key: &str) -> Option<String> {
        let (reply_tx, reply_rx) = oneshot::channel();
        self.tx.send(CacheMsg::Get { key: key.to_string(), reply: reply_tx }).await.ok()?;
        reply_rx.await.ok().flatten()
    }
}
```

Récapitulatif Jour 2

Points clés

- Mutex/RwLock : choix selon le pattern d'accès
- Atomics : orderings, CAS, compteurs lock-free
- Crossbeam : queues lock-free, work-stealing, epoch GC
- Pattern Actor avec `mpsc` / `oneshot`

Demain : Systèmes Distribués

